Maktaba Project Assignment

Assignment Overview

Title: Maktaba – Subscription-Based Learning Platform
Stack: Angular (Frontend), Node.js + Express (TypeScript) as Main Backend, Flask (Python) as AI Microservice, MySQL/MSSQL (Database), Docker (Containerization)

Context

You are building a modern learning platform where users can browse and subscribe to premium educational content. The system includes a Node.js + Express backend for core APIs, a Flask service for AI-driven course recommendations, and an Angular frontend for a sleek, responsive user experience. All components are containerized with Docker for seamless deployment.

Core Functional Requirements

1. User Registration & Authentication: Users can register, login, and manage their sessions securely.
2. Course Management: Admins can create, update, and delete courses. Courses can be tagged as free or premium.
3. Subscription & Payments: Users can subscribe to premium courses using Paystack or Daraja API.
4. AI Recommendations (Flask): Suggests personalized courses based on user behavior and history.
5. Pagination: Implemented on course lists, user lists, and subscription histories.
6. Rate Limiting: Protects APIs from abuse using libraries such as express-rate-limit.
7. Session Management: Secure session handling using JWT and Redis cache for session persistence.

UI Requirements

1. Dashboard – Displays user's enrolled courses and recommendations.
2. Course Explorer – Lists all courses with pagination and filters.
3. Course Details Page – Shows course info and 'Subscribe' button for premium content.
4. Admin Panel – Manage users, courses, and subscriptions.
5. Payments Page – Integrates Paystack or Daraja for transactions.

Backend Requirements

1. Node.js (TypeScript) RESTful API with Express.
2. Flask microservice for AI recommendations (served via internal Docker network).
3. MySQL/MSSQL database integration with Sequelize or TypeORM.

4. Environment variables for configuration (dotenv).
5. Docker Compose setup for Node, Flask, and Database containers.

API Endpoints

- POST /auth/register – Register a new user
- POST /auth/login – Authenticate user and issue JWT
- GET /courses – List all courses (paginated)
- POST /courses – Create a new course (Admin only)
- POST /payments/subscribe – Initiate payment for premium course
- GET /subscriptions – List user subscriptions
- GET /recommendations – Get AI-powered course suggestions (Flask)

Database Schema (MySQL/MSSQL)

- Users (id, name, email, password_hash, role, created_at)
- Courses (id, title, description, category, price, access_type, created_at)
- Subscriptions (id, user_id, course_id, payment_status, start_date, end_date)
- Payments (id, user_id, course_id, amount, payment_method, status, created_at)

Deliverables

1. GitHub repository with both backend (Node + Flask) and frontend code.
2. Docker Compose configuration for full app setup.
3. README.md with setup instructions for each service.
4. SQL or migration files for database setup.
5. Screenshots or short video of running system (optional).

Evaluation Criteria

1. Correctness – Meets all functional and technical requirements.
2. Code Quality – Clean, modular, and well-documented.
3. Security – Proper session handling, validation, and rate limiting.
4. UI/UX – Simple, consistent, and responsive interface.
5. Containerization – Proper use of Docker and Docker Compose.

Timeline

Recommended Duration: 7–10 Days

Bonus (Optional)

1. Add AI-based course summaries using OpenAI API via Flask.

2. Implement refresh tokens for extended sessions.
3. Add email verification and password reset flows.
4. Integrate caching for course listings using Redis.
5. Implement CI/CD with GitHub Actions or Docker Hub.